基于LangGraph的多智能体阿瓦隆桌游系统

一、实验题目

尝试利用开源的框架（推荐LangGraph、可以不造轮子）设计一个多智能体系统，要求：

- 场景/任务不限（任务求解、场景模拟、效率工具等）
- 包含Agent推理/规划方法的实现（ToT、ReAct、Reflection 等）
- 设计2种以上记忆机制（长期记忆、短期记忆）
- 使用2种以上的工具（rag检索、搜索引擎、计算工具等）也可自己定义工具。

评分标准：

1. 多智能体系统中智能体数目>=4：

- 每个智能体的功能有良好定位
- 每个智能体应该有适宜的提示词适配自己的任务
- 智能体间分工和运行流程合理

2. 在系统中应至少实现一种智能体的推理/规划方法
3. 在系统中设计智能体的长期记忆和短期记忆
4. 在系统中至少使用两种类型以上的工具调用

二、实验内容

根据实验题目的要求，我选择了实现一个基于LangGraph的多智能体阿瓦隆桌游系统，下面展示具体的算法、代码等。

2.1 算法原理

2.1.1 Agent推理/规划方法

ReAct（Reasoning and Acting)

核心思想：结合思考（Reasoning）与行动（Acting），让模型在推理过程中交替执行行动（例如查询搜索引擎、调用工具）并进行思考。

流程：

- 模型每一步会输出两个部分：
  - `Thought`：思考内容
  - `Action`：采取的行动（如检索、计算）
- 行动的结果（Observation）会被用于下一步推理。

优点：

- 允许模型与外部环境交互（例如用计算器或搜索）
- 提升了复杂任务（如问答、数学推理）的准确率

ToT（Tree of Thoughts)

核心思想：将模型的推理过程组织为一棵思维树（tree of thoughts），而不是线性推理流程。这样可以探索多个思路、分支，并进行筛选与评估。

流程：

1. 将复杂问题分解为多个决策步骤（如一个解题流程的多个子阶段）
2. 在每个阶段，生成多个不同的“思维”（thoughts）作为候选分支
3. 使用评分函数或模型自评机制评估这些思维的价值
4. 保留好的分支继续向下扩展，最终选出最优解

优点：

- 更好地探索解题空间
- 模仿人类“多想几步再做决定”的思维方式
- 适用于需要创意、多步骤推理的问题（如数独、代码生成等）

Reflection（自我反思）

核心思想：让语言模型在完成任务后进行「自我反思」，分析失败或成功原因，并用这些反思来指导下一轮尝试，从而逐步改进表现。

流程：

1. 模型尝试解决任务
2. 若失败，生成一段**反思文字**（Reflection），说明失败原因
3. 将反思作为上下文的一部分重新尝试
4. 重复迭代，逐步提高表现

优点：

- 不依赖额外数据或外部反馈机制
- 模拟人类“从错误中学习”的过程
- 提高模型长期任务表现，如代码调试、写作等

2.1.2 长期记忆与短期记忆

类型	内容	生命周期	存储位置
短期记忆	当前对话上下文、最近几轮交流等	当前会话周期	缓存在 memory buffer（如 ConversationBufferMemory）
长期记忆	过去的重要事件、知识、决策记录	永久或跨多个会话	存在数据库或向量存储中（如 Chroma, FAISS, Pinecone 等）

2.1.3 工具

RAG 检索（Retrieval-Augmented Generation）

RAG 是一种结合 **检索（Retrieval）+ 生成（Generation）** 的技术，先从知识库中检索相关内容，再由语言模型生成回答。

适合场景：

- 提问领域知识（如游戏规则、公司政策、法律、历史）；
- 想让模型只用“提供的文档”回答，不产生幻想内容。

搜索引擎工具

搜索引擎工具通过 API 调用真实的互联网搜索结果（例如 Google、Tavily、Bing 等），并将其作为信息源供模型参考。

适合场景：

- 获取实时信息；
- 查找互联网上的数据，如新闻、天气、人物等。

计算工具（如概率分析、投票模式分析）

适合场景：

- 分析团队组成的成功率；
- 判断投票是否一致、是否存在欺骗行为；
- 统计行为频率、胜率等。

2.2 阿瓦隆桌游规则介绍

为了方便读者能够更好地理解后续代码的作用，下面简单介绍一下阿瓦隆桌游规则：

《阿瓦隆》是一款考验玩家推理、策略和合作的桌游，玩家分为好人和坏人两个阵营，通过完成任务和投票来互相猜测和对抗。

2.2.1 游戏角色：

- **好人阵营：**

  梅林：好人阵营的主公，他能看到除了黑老大莫甘娜之外的所有坏人，他需要在游戏中隐藏自己，给派西维尔透露信息。

  派西维尔：好人阵营的带队人，他能看到莫甘娜和梅林，但不知道谁是真正的梅林，他需要确定真正的梅林，然后带领好人阵营组队赢得游戏。

  忠臣：好人阵营的成员，他们只能看到自己的身份牌，需要跟随派西维尔的领导让任务成功，并且隐藏真正的梅林身份。

- **坏人阵营：**

  莫甘娜：坏人阵营的领头人，莫甘娜能看见除奥伯伦之外的坏人，同时他能被梅林和派西维尔看见，但派西维尔并不知道他的真正身份，所以他可以装作自己是梅林来迷惑派西维尔。

  刺客：坏人方的角色，他能被梅林看见，他的任务是寻找真正的梅林，并在好人赢得三局游戏后刺杀梅林，若刺杀成功则坏人胜利。

  莫德雷德：在忠臣里的坏人，梅林晚上看不到他，所以他隐藏在忠臣里，混进队伍让任务失败。

  奥伯伦：坏人方角色，但是他看不到他的队友，他的队友也看不到他，他能被梅林看见。

  爪牙：坏人方角色，能被梅林看到。

2.2.2 游戏目标:

- **好人阵营:**

  成功完成3个任务，并隐藏好人阵营中的重要人物梅林，避免他被刺杀。

- **坏人阵营:**

  阻止好人阵营完成3个任务，或在游戏结束时成功刺杀梅林，以此取得游戏胜利。

2.2.3 游戏人数与游戏角色配置

游戏人数	好人阵营	坏人阵营
6	梅林、派西维尔、2个忠臣	莫甘娜、刺客
7	梅林、派西维尔、2个忠臣	莫甘娜、刺客、奥伯伦
8	梅林、派西维尔、3个忠臣	莫甘娜、刺客、爪牙
9	梅林、派西维尔、4个忠臣	莫甘娜、刺客、黑老大
10	梅林、派西维尔、4个忠臣	莫甘娜、刺客、黑老大、奥伯伦

由于本次实验的重点并不在于这个游戏本身，因此为了简便起见，我在实验中选取了6个人的角色配置。其中**有5个是智能体**，1个是人类玩家。

之后有时间可以增加人数，完善游戏规则，把这个实验转变为一个简易的桌游软件。

2.3 关键代码展示

2.3.1 ReAct（Reasoning and Acting)

在该系统中，主要采用的Agent推理/规划方法是**ReAct**，具体实现如下:

1.在系统提示中就明确给要求Agent可以使用三种策略来思考

```
def _get_system_prompt(self) -> str:
    return f"""
    你现在正在参与阿瓦隆游戏。
    你是一个阿瓦隆游戏中的{self.role}角色。
    你的目标是{'帮助好人阵营完成任务' if self.role in ['Merlin', 'Percival', 'Loyal
Servant1','Loyal Servant2'] else '破坏任务并隐藏身份'}。
    你的特殊能力: {ROLES[self.role]}
    游戏当前阶段: {{current_round}}/5轮
    当前任务团队: {{mission_team}}
    已知信息: {{summary}}

    你可以使用以下策略:
    1. ToT (思维树): 当需要复杂决策时，考虑多种可能性
    2. ReAct (推理+行动): 结合推理和工具使用
    3. Reflection: 回顾游戏历史分析其他玩家行为

    请谨慎行动，不要暴露你的身份。
    ...
    """
```

2.代理执行器（AgentExecutor）初始化时使用了ReAct Agent类型

```
self.executor = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    memory=self.memory_short,
    handle_parsing_errors=True,
    verbose=True
)
```

`ZERO_SHOT_REACT_DESCRIPTION`是一个基于**ReAct**思想的零样本Agent实现，能够让模型结合思考和调用工具。

3.Agent行为方法 `act` 调用executor，形成推理-行动循环

```
def act(self, state: AgentState) -> Dict:
    if isinstance(state, str):
        state = json.loads(state)
    if self.is_human:
        input_msg = state["messages"][-1]["content"]
        human_input = input(f"{self.name}({self.role}), 请输入你的发言:
{input_msg}\n> ")
        return {"messages": [HumanMessage(content=human_input)]}

    # 更新记忆
    for msg in state["messages"]:

        if msg["role"] == "human":
            self.memory_short.chat_memory.add_user_message(str(msg["content"]))
        else:
            self.memory_short.chat_memory.add_ai_message(str(msg["content"]))

    # 准备输入
    last_msg = state["messages"][-1]["content"]
    input_dict = {
        "input": last_msg,
        "votes": state.get("votes", []),
        "current_round": state["current_round"],
        "mission_team": ", ".join(state["mission_team"]),
        "intermediate_steps": []
    }
    # 调用智能体（Agent），根据输入进行决策
    response = self.executor.invoke(input_dict)

    # 更新长期记忆
    self.memory_long.save_context(
        {"input": last_msg},
        {"output": response["output"]}
    )

    return {"messages": [AIMessage(content=response["output"])]}
```

2.3.2 长期记忆与短期记忆

在Agent基础类中定义短期记忆和长期记忆

```
class AvalonAgent:
    def __init__(self, name: str, role: str, is_human: bool = False):
        self.name = name
        self.role = role
        self.is_human = is_human
        self.memory_short = ConversationBufferMemory(memory_key="chat_history",
return_messages=True)  # 短期记忆
        self.memory_long = ConversationSummaryMemory(llm=llm,
memory_key="summary")  # 长期记忆
```

在act函数中，对短期记忆和长期记忆进行更新：

对于短期记忆（`memory_short`）

- 暂时保存当前对话会话中的消息，方便智能体即时参考上下文。

对于长期记忆（`memory_long`）

- 调用 `save_context` 方法，将本次交互的输入和输出存储起来。方便后续会话或者跨会话调用。

```
def act(self, state: AgentState) -> Dict:
    if isinstance(state, str):
        state = json.loads(state)
    if self.is_human:
        input_msg = state["messages"][-1]["content"]
        human_input = input(f"{self.name}({self.role}), 请输入你的发言:
{input_msg}\n> ")
        return {"messages": [HumanMessage(content=human_input)]}

    # 更新短期记忆
    for msg in state["messages"]:
        if msg["role"] == "human":
            self.memory_short.chat_memory.add_user_message(str(msg["content"]))
        else:
            self.memory_short.chat_memory.add_ai_message(str(msg["content"]))

    # 准备输入
    last_msg = state["messages"][-1]["content"]
    input_dict = {
        "input": last_msg,
        "votes": state.get("votes", []),
        "current_round": state["current_round"],
        "mission_team": ", ".join(state["mission_team"]),
        "intermediate_steps": []
    }
    # 调用智能体（Agent），根据输入进行决策
    response = self.executor.invoke(input_dict)

    # 更新长期记忆
    self.memory_long.save_context(
        {"input": last_msg},
        {"output": response["output"]}
```

```
    )

    return {"messages": [AIMessage(content=response["output"])]}
```

2.3.3 工具使用

我总共使用了4种工具，分别是：**RAG检索工具**、**Tavily搜索引擎**、**团队任务成功概率计算工具**以及**投票分析工具**

RAG检索工具

该工具在这个系统中主要用于查询游戏规则，具体代码如下：

```
# 创建RAG检索工具
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=200)
avalon_rules = """
阿瓦隆游戏规则：
1. 游戏分为好人阵营和坏人阵营
2. 好人包括梅林、派西维尔和2位忠臣
3. 坏人包括刺客和莫甘娜
4. 游戏进行5轮任务，每轮需要组队执行
5. 团队需要投票通过后才能执行任务
6. 坏人可以 sabotage 任务
7. 游戏结束时刺客可以刺杀梅林

各个玩家的能力：
梅林　知道所有坏人（除了莫德雷德，如有）身份，但不能暴露自己
派西维尔　　知道“梅林”和“莫甘娜”的两个候选人（无法区分谁是谁）
莫甘娜　冒充梅林，出现在派西维尔面前
刺客　若游戏进入刺杀阶段，有一次刺杀梅林的机会
忠臣　无特殊能力，通过逻辑和行为判断他人身份

胜利条件：
好人阵营胜利条件：
成功完成3次任务，且梅林未被刺杀
坏人阵营胜利条件：
成功破坏3次任务，或成功识破并刺杀梅林

战术建议
对于好人：
梅林需要巧妙引导投票与任务，但不能太明显；
派西维尔需要保护梅林并识破莫甘娜；
忠臣需认真分析行为逻辑、投票模式、发言内容；

对于坏人：
卧底应表现得像好人，避免暴露身份；
投票时制造分歧、引导失败任务；
刺客要利用游戏历史，准确识别梅林。
"""
documents = text_splitter.create_documents([avalon_rules])
vectorstore = FAISS.from_documents(documents, embeddings)  # 使用DashScope嵌入模型
retriever = vectorstore.as_retriever()
```

```
def rag_retriever(query: str) -> str:
    """RAG检索工具函数"""
    docs = retriever.get_relevant_documents(query)
    return "\n\n".join(doc.page_content for doc in docs)

tools.append(
    Tool(
        name="AvalonRulesRetriever",  # RAG检索工具
        func=rag_retriever,
        description="检索阿瓦隆游戏规则和策略"
    )
)
```

Tavily搜索引擎

引入搜索引擎是为了当Agent需要知道除了已经给定的规则之外的其他规则或者是游戏策略时，可以查询搜索引擎。

```
from langchain_tavily import TavilySearch
Tool(
    name="TavilySearch",
    func=TavilySearch(max_results=2),
    description="使用Tavily搜索引擎获取最新信息"
)
```

团队任务成功概率计算工具

团队任务成功概率计算工具用于计算任务成功概率。

```
def calculate_mission_success_probability(mission_team: List[str]) -> str:
    """计算任务成功概率的工具"""
    good_players = sum(1 for p in mission_team if p in ["Merlin", "Percival",
"Loyal Servant1","Loyal Servant2"])
    bad_players = sum(1 for p in mission_team if p in ["Assassin", "Morgana",
"Mordred"])
    success_prob = 0.8 - 0.2 * bad_players + 0.1 * good_players
    return f"任务成功概率: {min(max(success_prob, 0), 1):.0%}"


Tool(
        name="MissionSuccessCalculator",
        func=calculate_mission_success_probability,
        description="计算给定任务团队的成功概率"
    )
```

投票分析工具

定义投票分析工具，帮助AI分析投票情况，从而分析其他玩家潜在的行为

```
def analyze_voting_patterns(input: str) -> str:
    """分析投票模式工具"""
    import json
    try:
        votes_data = json.loads(input)
        votes = votes_data.get("votes", [])
```

```
        if not votes:
            return "没有投票数据。"

        approvals = [v for v in votes if v["vote"].lower() in ("yes", "approve",
"true")]
        opposers = [v for v in votes if v["vote"].lower() in ("no", "reject",
"false")]

        result = f"共有 {len(votes)} 名玩家参与投票。\n"
        result += f"赞成人数: {len(approvals)}，反对人数: {len(opposers)}。\n"

        if len(opposers) > len(approvals):
            result += "团队意见分歧较大，可能存在阵营对立。"
        elif len(opposers) == 0:
            result += "全员一致赞成，团队高度一致。"
        else:
            result += "团队中存在部分不信任或怀疑。"

        return result
    except Exception as e:
        return f"投票分析失败: {e}"

Tool(
        name="VotingPatternAnalyzer",
        func=analyze_voting_patterns,
        description="分析投票模式以识别潜在行为模式",
    )
```

2.3.4 Agent

Agent状态定义如下：

```
# 定义Agent状态
class AgentState(TypedDict):
    messages: List[Dict]
    memory: Dict[str, any]
    current_round: int
    mission_team: List[str]
    votes: Dict[str, bool]
    game_history: List[Dict]
```

Agent基础类定义如下：

```
# 定义Agent基础类
class AvalonAgent:
    def __init__(self, name: str, role: str, is_human: bool = False):
        self.name = name
        self.role = role
        self.is_human = is_human
        self.memory_short = ConversationBufferMemory(memory_key="chat_history",
return_messages=True)  # 短期记忆
        self.memory_long = ConversationSummaryMemory(llm=llm,
memory_key="summary")  # 长期记忆
```

```
        # 创建Agent执行器
        prompt = ChatPromptTemplate.from_messages([
            ("system", self._get_system_prompt()),
            MessagesPlaceholder(variable_name="chat_history"),
            ("human", "{input}"),
            MessagesPlaceholder(variable_name="agent_scratchpad"),
        ])

        agent = (
                RunnablePassthrough.assign(
                    agent_scratchpad=lambda x:
format_log_to_str(x["intermediate_steps"]),
                    chat_history=lambda x:
self.memory_short.load_memory_variables({})["chat_history"],
                    summary=lambda x: self.memory_long.load_memory_variables({})
["summary"],
                )
                | prompt
                | llm.bind(stop=["\nObservation:"])
                | JSONAgentOutputParser()
        )

        self.executor = initialize_agent(
            tools=tools,
            llm=llm,
            agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
            memory=self.memory_short,
            handle_parsing_errors=True,
            verbose=True
        )

    def _get_system_prompt(self) -> str:
        ...

    def act(self, state: AgentState) -> Dict:
        ...
```

2.3.5 游戏场景

在游戏中，我定义了5个Agent，1个人类玩家，分别扮演不同的角色

```
# 创建游戏模拟器
class AvalonGameSimulator:
    def __init__(self):
        # 初始化5个智能体
        self.agents = [
            AvalonAgent("agent1", "Merlin"),
            AvalonAgent("agent2", "Percival"),
            AvalonAgent("agent3", "Loyal Servant1"),
            AvalonAgent("agent4", "Morgana"),
            AvalonAgent("agent5", "Assassin")
        ]

        # 初始化1个玩家
```

```
        self.user = AvalonAgent("玩家", "Loyal Servant2", is_human=True)

        # 初始化游戏状态
        self.game_state = {
            "current_round": 1,
            "mission_team": [],
            "votes": {},
            "game_history": [],
            "good_wins": 0,
            "evil_wins": 0
        }
```

游戏的主要代码如下：

```
def start_game(self):
    print("阿瓦隆游戏开始!")
    self.assign_roles()

    # 游戏主循环 (5轮任务)
    for round_num in range(1, 6):
        self.game_state["current_round"] = round_num
        print(f"\n=== 第 {round_num} 轮任务 ===")

        # 1. 组队阶段
        self.team_selection_phase()
        # 2. 轮流发言阶段
        self.speaking_phase()
        # 3. 投票阶段
        if not self.voting_phase():
            continue  # 投票未通过，重新组队

        # 4. 任务执行阶段
        self.mission_execution_phase()

        # 检查游戏结束条件
        if self.game_state["good_wins"] >= 3 or self.game_state["evil_wins"] >=
3:
            break

    # 游戏结束，刺客刺杀梅林阶段
    self.assassination_phase()
```

2.4 创新点

2.4.1 在系统提示中集成策略方法

在 `system prompt` 中明确提出了三种认知行为策略：

- ReAct（推理+行动）
- ToT（树状思考/多路径推理）
- Reflection（回顾历史自我评估）

这种混合思维范式的提示词工程设计，有助于Agent利用不同策略思考，生成更高级的推理内容。

```
def _get_system_prompt(self) -> str:
    return f"""
    你现在正在参与阿瓦隆游戏。
    你是一个阿瓦隆游戏中的{self.role}角色。
    你的目标是{'帮助好人阵营完成任务' if self.role in ['Merlin', 'Percival', 'Loyal
Servant1','Loyal Servant2'] else '破坏任务并隐藏身份'}。
    你的特殊能力: {ROLES[self.role]}
    游戏当前阶段: {{current_round}}/5轮
    当前任务团队: {{mission_team}}
    已知信息: {{summary}}

    你可以使用以下策略:
    1. ToT (思维树): 当需要复杂决策时，考虑多种可能性
    2. ReAct (推理+行动): 结合推理和工具使用
    3. Reflection: 回顾游戏历史分析其他玩家行为

    请谨慎行动，不要暴露你的身份。
    ...
    """
```

2.4.2 容错率高

在下面的`executor`中，加入`handle_parsing_errors=True`，可以使得对于agent不符合格式的回答，可以自动重新回答，提高了系统的容错率，避免代码因为格式不对而报错。

```
self.executor = initialize_agent(
        tools=tools,
        llm=llm,
        agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
        memory=self.memory_short,
        handle_parsing_errors=True,
        verbose=True
    )
```

2.4.3 交互性强

实现了人类玩家与Agent的交互

1.在轮流发言阶段，人类玩家可以输入发言

```
def speaking_phase(self):
    self.broadcast_message("现在进入轮流发言阶段，请按顺序发表对局势的看法")
    all_players = self.agents + [self.user]
    for i, player in enumerate(all_players):
        # 通知当前玩家发言
        if player.is_human:
            # 人类玩家输入发言
            print(f"\n{player.name}({player.role})，请发言（{i +
1}/{len(all_players)}）")
            print("请分析当前局势，推测好人坏人身份")
            speech = input("你的发言内容: ")
        else:
            # AI玩家自动生成发言
            response = player.act({
```

```
                "messages": [{
                    "role": "system",
                    "content": f"请发言（{i + 1}/{len(all_players)}），分析当前局势，
推测好人坏人"
                }],
                "current_round": self.game_state["current_round"],
                "mission_team": self.game_state["mission_team"],
                "votes": self.game_state.get("votes", {}),
                "game_history": self.game_state["game_history"]
            })
            speech = response["messages"][0].content
            print(f"\n{player.name}({player.role}) 发言: {speech}")

        # 广播该玩家的发言给所有人
        self.broadcast_message({
            "type": "speech",
            "player": player.name,
            "content": response["messages"][0].content,
            "to": "all"
        })

        # 记录发言到游戏历史
        self.game_state["game_history"].append({
            "round": self.game_state["current_round"],
            "phase": "speaking",
            "player": player.name,
            "content": speech
        })
```

2.在投票环节，人类玩家也可以参与投票

```
# 收集所有玩家投票
for agent in self.agents + [self.user]:
    if agent.is_human:
        vote = input(f"{agent.name}, 你是否赞成当前任务团队? (y/n): ")
        self.game_state["votes"][agent.name] = vote.lower() == 'y'
    else:
        # AI根据角色和游戏历史决定投票
        reasoning = ("作为好人，我信任这个团队" if agent.role in ["Merlin",
"Percival", "Loyal Servant1","Loyal Servant2"]
                     else "作为坏人，我需要让团队看起来可疑")
        vote = random.random() > 0.3 if agent.role in ["Merlin", "Percival",
                                                       "Loyal Servant"] else
random.random() > 0.7
        self.game_state["votes"][agent.name] = vote
        print(f"{agent.name}({agent.role}) 投票: {'赞成' if vote else '反对'} -
{reasoning}")
```

通过这两种方法，使得系统具有较强的交互性，能够和人类玩家一起进行这个桌游。

2.4.4 特定工具

依据游戏场景需要，我设计了特定的工具：

- `VotingPatternAnalyzer`：用于分析行为模式
- `MissionSuccessCalculator`：计算队伍组成影响任务结果的概率
- `AvalonRulesRetriever`： 结合RAG检索阿瓦隆规则和战术

具体代码在“关键代码展示”中已经展示，这里就不再重复。

2.4.5 将Agent应用于社交推理类游戏

ReAct 通常应用于问答等任务，而把它使用到阿瓦隆这类社交推理类游戏中比较创新。能够发挥ReAct 推理加行动的优势。

三、实验结果及分析

下面展示部分在游戏过程中，Agent的表现效果：（但是在作业提交的那天（即7月6日），我的API-Key用完了，所以没有展示更多的效果）

```
> Entering new AgentExecutor chain...
我需要分析当前的局势，并推测好人和坏人的可能角色。首先，我需要了解当前的游戏状态和团队成员的表现。

Action: TavilySearch
Action Input: "阿瓦隆游戏局势分析 好人坏人推测"
Observation: {'query': '阿瓦隆游戏局势分析 好人坏人推测', 'follow_up_questions': None, 'answer': None, 'images': [], 'results': [{'url': 'https://www.zhihu.com/question/35383437/answer/3196831167', 'title': '《阿瓦隆》有哪些心得、攻略？ - 黑面高人的回答- 知乎', 'content': '换位思考是这个游戏的精髓之一，很多的软逻辑是可以靠换位思考推理出来。甚至可以引申出好人和坏人之间的心理战。 换位思考具体指，设身处地假设自己是场上某个其他', 'score': 0.568057, 'raw_content': None}, {'url': 'https://www.reddit.com/r/boardgames/comments/ph765d/insights_from_10000_games_of_the_resistance_avalon/?tl=zh-hans', 'title': '来自10000局抵抗组织：阿瓦隆游戏的见解：r/boardgames - Reddit', 'content': '如果好人玩家设法从第一个任务开始连续通过3个任务来横扫游戏，梅林只有大约1/3的几率被杀。但如果好人玩家需要5个任务才能获得3次胜利，梅林被杀的几率几乎', 'score': 0.54971373, 'raw_content': None}], 'response_time': 1.7}
Thought:根据搜索结果，阿瓦隆游戏的核心在于推理和心理战。好人和坏人的角色可以通过任务成功与否、投票模式以及玩家之间的互动进行推测。

在当前局势中，如果任务团队的组成和投票模式显示出某些一致性或异常行为（例如，某人总是被选中但任务失败），这可能表明该玩家是坏人。反之，如果某个玩家的行为符合逻辑且有助于任务成功，则可能是好人。

最终答案：在阿瓦隆游戏中，好人和坏人的角色通常通过任务成功率、投票行为和逻辑推理来推测。如果任务团队的表现不一致或存在可疑行为，可能暗示坏人的存在；而逻辑清晰、协作良好的玩家更可能是好人。具体分析需要结合当前的游戏状态和玩家表现。
Observation: Invalid Format: Missing 'Action:' after 'Thought:
Thought: 我需要分析当前的局势，并推测好人和坏人的可能角色。首先，我需要了解当前的游戏状态和团队成员的表现。
```

```
> Entering new AgentExecutor chain...
```

当前问题涉及对游戏局势的分析，需要结合游戏规则和玩家行为模式。我需要进一步了解阿瓦隆游戏的规则以及如何通过投票记录和发言内容来识别好人和坏人。

Action: AvalonRulesRetriever
Action Input: 识别好人和坏人的方法
Observation: 阿瓦隆游戏规则:
1. 游戏分为好人阵营和坏人阵营
2. 好人包括梅林、派西维尔和2位忠臣
3. 坏人包括刺客和莫甘娜
4. 游戏进行5轮任务，每轮需要组队执行
5. 团队需要投票通过后才能执行任务
6. 坏人可以 sabotage 任务
7. 游戏结束时刺客可以刺杀梅林

各个玩家的能力：
梅林　知道所有坏人（除了莫德雷德，如有）身份，但不能暴露自己
派西维尔　　知道“梅林”和“莫甘娜”的两个候选人（无法区分谁是谁）
莫甘娜 冒充梅林，出现在派西维尔面前
刺客　若游戏进入刺杀阶段，有一次刺杀梅林的机会
忠臣　无特殊能力，通过逻辑和行为判断他人身份

胜利条件：
好人阵营胜利条件：
成功完成3次任务，且梅林未被刺杀
坏人阵营胜利条件：
成功破坏3次任务，或成功识破并刺杀梅林

战术建议
对于好人：
梅林需要巧妙引导投票与任务，但不能太明显；
派西维尔需要保护梅林并识破莫甘娜；
忠臣需认真分析行为逻辑、投票模式、发言内容；

对于坏人：
卧底应表现得像好人，避免暴露身份；
投票时制造分歧、引导失败任务；
刺客要利用游戏历史，准确识别梅林。
Thought:Thought: 从规则中了解到，好人需要通过观察玩家行为和投票记录来识别坏人。我需要进一步分析投票模式以推测可能的坏人身份。

Action: VotingPatternAnalyzer
Action Input: 分析投票模式以识别潜在行为模式
Observation: 投票分析失败: Expecting value: line 1 column 1 (char 0)
Thought:Thought: 投票模式分析工具的输入格式可能不正确。我需要重新尝试提供合适的输入。

Action: VotingPatternAnalyzer
Action Input: 请分析投票记录以识别潜在行为模式
Observation: 投票分析失败: Expecting value: line 1 column 1 (char 0)
Thought:Thought: 投票模式分析工具似乎无法处理当前输入。可能需要更具体的投票记录数据。我需要尝试使用其他方法，例如通过搜索获取最新信息或规则。

Action: TavilySearch
Action Input: 阿瓦隆游戏如何通过投票和发言识别好人和坏人

```
Observation: {'query': '阿瓦隆游戏如何通过投票和发言识别好人和坏人',
'follow_up_questions': None, 'answer': None, 'images': [], 'results': [{'url':
'https://zhuanlan.zhihu.com/p/422741835', 'title': '阿瓦隆玩法 - 知乎专栏',
'content': '首先这个游戏分为好人阵营和坏人阵营（卡牌右上角蓝色为好人，红色为坏人阵营 ... 投票
选择第一个队长，若平票则pk发言。 队长职责：. 队长负责先预选做任务的人员', 'score':
0.7859175, 'raw_content': None}, {'url': 'https://m.douban.com/note/523559795/',
'title': '桌游干货《阿瓦隆》游戏规则及速成攻略 - 豆瓣', 'content': '* 真梅林在发言阶段避重就
轻、似是而非地说出情报，但不能一回合点中全部好人/坏人，可以一回合点1-2个甚至故意点错（蒙骗刺
客），再由下一回合更正。 ... * 忠臣', 'score': 0.61251885, 'raw_content': None}],
'response_time': 1.02}
Thought:根据搜索结果，阿瓦隆游戏的规则中提到，好人和坏人的身份可以通过投票记录、发言内容以及任务
执行情况来推测。例如：

1. **梅林**是唯一能识别莫甘娜（坏人）的好人，但需要通过发言和行为来判断。
2. **派西维尔**可以知道梅林的身份，但不能直接透露。
3. **忠臣**通常会支持梅林的判断，并配合完成任务。
4. **坏人**（如莫甘娜和刺客）会伪装成好人，可能误导投票或在任务中破坏。

此外，投票模式分析可以帮助识别潜在行为模式，例如：
- 如果某位玩家经常反对合理的任务组队，可能是坏人。
- 如果某位玩家的发言与实际任务结果不一致，可能有隐藏身份。

综上所述，结合游戏规则和观察玩家行为是识别好人和坏人的重要方法。建议继续关注玩家的发言和投票行为，
以做出更准确的判断。

Final Answer: 在阿瓦隆游戏中，识别好人和坏人需要结合游戏规则和玩家行为。好人包括梅林、派西维尔
和忠臣，他们通过合作完成任务并识别坏人；而坏人（如莫甘娜和刺客）会伪装并误导好人。通过分析投票记
录、发言内容和任务执行情况，可以推测玩家身份。例如，坏人可能在投票中表现出异常行为，或在发言中提供
误导性信息。因此，持续观察玩家的行为和互动是关键。

> Finished chain.
```

由输出可见，Agent运用了ReAct的推理方法，使用了**RAG检索工具**来查询游戏规则、使用**Tavily搜索引擎**来搜索游戏制胜的策略、使用**投票分析工具**来分析其他玩家潜在的行为。

不足之处：

1. 目前的游戏逻辑没有全部实现，比较复杂的游戏规则尚未实现。
2. 在作业提交的那天（即7月6日），我的API-Key用完了，导致代码中还有一些小问题无法调试并解决。

四、参考文献

[1] 概述 - LangChain 框架

[2] https://www.cnblogs.com/Ethereal-hzc/p/18639165